

Bash scripting

- A **script** is a text file of shell commands that runs as a single program.

- A **script** is a text file of shell commands that runs as a single program.
- Scripts turn multi-step terminal work into **repeatable, shareable** procedures.

```
nano run_analysis.sh
```

Creating and running a script

```
nano run_analysis.sh  
  
#!/bin/bash  
echo "Starting analysis..."  
python src/analyze.py  
echo "Done."
```

Creating and running a script

```
nano run_analysis.sh

#!/bin/bash
echo "Starting analysis..."
python src/analyze.py
echo "Done."

chmod +x run_analysis.sh
./run_analysis.sh
```

Creating and running a script

```
nano run_analysis.sh

#!/bin/bash

echo "Starting analysis..."
python src/analyze.py
echo "Done."

chmod +x run_analysis.sh
./run_analysis.sh
```

- **#!/bin/bash** (the shebang) tells the OS which interpreter to use.

Creating and running a script

```
nano run_analysis.sh

#!/bin/bash
echo "Starting analysis..."
python src/analyze.py
echo "Done."

chmod +x run_analysis.sh
./run_analysis.sh
```

- **#!/bin/bash** (the shebang) tells the OS which interpreter to use.
- **chmod +x** makes the file executable.

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

Variables

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

- Assign with no spaces around =.

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

- Assign with **no spaces** around =.
- Reference with **\$VAR** or **\${VAR}** (braces clarify boundaries).

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

- Assign with **no spaces** around =.
- Reference with **\$VAR** or **\${VAR}** (braces clarify boundaries).
- **\$(command)** — command substitution: run **command** and substitute its standard output as text. Use it on the **right-hand side** of = to store a result in a variable:

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

- Assign with **no spaces** around =.
- Reference with **\$VAR** or **\${VAR}** (braces clarify boundaries).
- **\$(command)** — command substitution: run **command** and substitute its standard output as text. Use it on the **right-hand side** of = to store a result in a variable:

```
NAME="experiment_01"  
echo "Running $NAME"  
echo "Output goes to output/${NAME}.csv"
```

- Assign with **no spaces** around =.
- Reference with **\$VAR** or **\${VAR}** (braces clarify boundaries).
- **\$(command)** — command substitution: run **command** and substitute its standard output as text. Use it on the **right-hand side** of = to store a result in a variable:

```
DAY=$(date +%Y-%m-%d)  
lines=$(wc -l < notes.txt)
```

```
if [ -f "data/raw.csv" ]; then
    echo "Raw data found."
elif [ -f "data/backup.csv" ]; then
    echo "Using backup."
else
    echo "No data available."
    exit 1
fi
```

```
if [ -f "data/raw.csv" ]; then
    echo "Raw data found."
elif [ -f "data/backup.csv" ]; then
    echo "Using backup."
else
    echo "No data available."
    exit 1
fi
```

- **-f** tests if a file exists; **-d** tests a directory.

```
if [ -f "data/raw.csv" ]; then
    echo "Raw data found."
elif [ -f "data/backup.csv" ]; then
    echo "Using backup."
else
    echo "No data available."
    exit 1
fi
```

- **-f** tests if a file exists; **-d** tests a directory.
- **exit 1** signals failure to the caller.

for loops

```
for file in data/*.csv; do
    echo "Processing $file"
    python src/clean.py "$file"
done
```

for loops

```
for file in data/*.csv; do
    echo "Processing $file"
    python src/clean.py "$file"
done
```

- Loops over every .csv in data/.

for loops

```
for file in data/*.csv; do
    echo "Processing $file"
    python src/clean.py "$file"
done
```

- Loops over every `.csv` in `data/`.
- Quote `"$file"` to handle filenames with spaces.

while loops

```
COUNT=1
while [ $COUNT -le 5 ]; do
    echo "Iteration $COUNT"
    ((COUNT++))
done
```

```
greet() {  
    echo "Hello, $1!"  
}
```

```
greet "World"  
greet "$USER"
```

```
greet() {  
    echo "Hello, $1!"  
}
```

```
greet "World"  
greet "$USER"
```

- `$1`, `$2`, ... are positional arguments to the function (or to the script when called from the command line).

- **read** — prompt for user input.

- **read** — prompt for user input.

- **read** — prompt for user input.

```
echo "Enter subject ID:"
```

```
read SUBJECT
```

```
echo "Processing $SUBJECT"
```

- **read** — prompt for user input.

```
echo "Enter subject ID:"
```

```
read SUBJECT
```

```
echo "Processing $SUBJECT"
```

- **date** — embed timestamps.

- **read** — prompt for user input.

```
echo "Enter subject ID:"
```

```
read SUBJECT
```

```
echo "Processing $SUBJECT"
```

- **date** — embed timestamps.

- **read** — prompt for user input.

```
echo "Enter subject ID:"  
read SUBJECT  
echo "Processing $SUBJECT"
```

- **date** — embed timestamps.

```
echo "Run started at $(date +%Y-%m-%d_%H:%M)"
```

- **read** — prompt for user input.

```
echo "Enter subject ID:"  
read SUBJECT  
echo "Processing $SUBJECT"
```

- **date** — embed timestamps.

```
echo "Run started at $(date +%Y-%m-%d_%H:%M)"
```

- **exit 0** — end the script with success status.

- Run with **bash -x script.sh** to print each command before execution.

- Run with **bash -x script.sh** to print each command before execution.
- Inside a script, toggle with **set -x** / **set +x**:

- Run with **bash -x script.sh** to print each command before execution.
- Inside a script, toggle with **set -x** / **set +x**:

- Run with **bash -x script.sh** to print each command before execution.
- Inside a script, toggle with **set -x** / **set +x**:

```
set -x
```

```
python src/fit_model.py
```

```
set +x
```

```
# This line is a comment  
echo "This line runs" # inline comment
```

A small real-world script

```
#!/bin/bash
# preprocess.sh – clean raw data and produce summary
set -e  # stop on first error
RAW="data/raw"
OUT="data/processed"
mkdir -p "$OUT"
for csv in "$RAW"/*.csv; do
    base=$(basename "$csv")
    python src/clean.py "$csv" > "$OUT/$base"
done
echo "Cleaned $(ls "$OUT" | wc -l) files."
```

- Shebang + **chmod +x** to create runnable scripts.

- Shebang + **chmod +x** to create runnable scripts.
- Variables, conditionals (`if/elif/else`), loops (`for, while`), functions.

- Shebang + **chmod** **+x** to create runnable scripts.
- Variables, conditionals (**if/elif/else**), loops (**for, while**), functions.
- **set -e** for fail-fast; **bash -x** for debugging.

- Shebang + **chmod +x** to create runnable scripts.
- Variables, conditionals (**if/elif/else**), loops (**for, while**), functions.
- **set -e** for fail-fast; **bash -x** for debugging.
- Scripts turn one-off CLI steps into reproducible pipelines.